

Async job processing architecture

A product team needs to accept long-running jobs over an API, run them in the background, retry failures, expose status, notify on completion or failure, and cancel work that is already running — including under high, spiky load. This is the system I would build.

NestJS

Azure Service Bus

MongoDB

Assumptions

- Multi-tenant SaaS. The caller is already authenticated; every job belongs to a tenant.
- A job is a machine-run unit of work: a type plus a payload. Large payloads live in blob storage; the queue message is only `jobId`.
- Delivery is at-least-once. Workers are idempotent. Exactly-once is not assumed.
- Cancel is cooperative. The worker checks a field on the job document. We do not kill the process or yank a locked Service Bus message.
- Notification is an HTTPS webhook supplied at submit time.
- Azure: AKS, Service Bus, MongoDB via Cosmos DB's Mongo API.

High-level architecture

Three moving parts. The API never runs the job. It writes a document, puts a message on the bus, and returns 202. Workers compete for those messages. MongoDB is the record of what happened: status, attempts, cancel, and the webhook outbox.

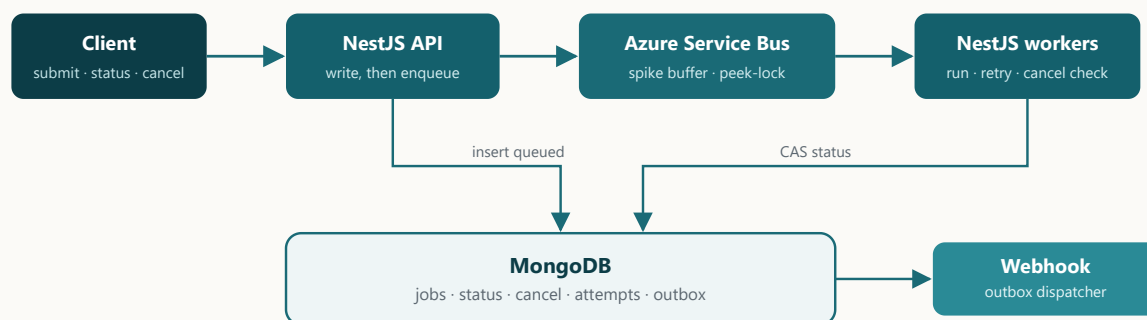


Figure 1. Request path and work path. The API records intent; workers do the work; MongoDB is the source of truth.

Happy path

- 1 `POST /jobs` validates the request, inserts a `queued` document, enqueues `jobId` on Service Bus, and returns `202`.
- 2 A worker peek-locks the message, conditionally sets the document to `running`, and executes the handler.
- 3 On success it sets `succeeded` and inserts an outbox document in the same transaction, then completes the message. A dispatcher POSTs the webhook.

Spikes sit on Service Bus. Workers scale on queue depth (KEDA). If the queue or a tenant is over a cap, submit returns 429 instead of taking unbounded work.

Major components

Job API · NestJS

Public surface: submit, get status, cancel. Auth, validation, tenant checks. Write the document first, then enqueue. If enqueue fails, a sweeper re-enqueues leftover `queued` jobs so nothing is stranded.

MongoDB

One document in `jobs` is the source of truth. Status moves as `queued` → `running` → `succeeded` | `failed` | `cancelled`. Updates use `findOneAndUpdate` with `status: expected` so two workers cannot start the same job. The `outbox` collection is written in the same transaction as a terminal status.

Azure Service Bus

The spike buffer. Peek-lock, max delivery count, dead-letter queue, duplicate detection, scheduled messages for backoff. One queue to start. The message body is small: `jobId`, `type`, `attempt`.

Workers · NestJS, second process

Same codebase as the API. Pull from the bus. Skip the job if it is already terminal or cancel was requested. Run the handler. Complete the message when done. Check `cancelRequestedAt` between steps and on lock renewal.

Webhook dispatcher. Reads the outbox, POSTs a signed payload, retries. The job is already terminal; a down webhook does not re-run work. Clients can always `GET /jobs/:id`.

Job lifecycle

Status is a state machine, not a free-form string. Terminal states never leave that state. `running` → `queued` is a `retry`: `attempt` increments, and `backoff` is a scheduled Service Bus message, not a timer inside the worker. `cancelRequestedAt` is a flag. Status becomes `cancelled` only when the API (still `queued`) or the worker applies the transition.

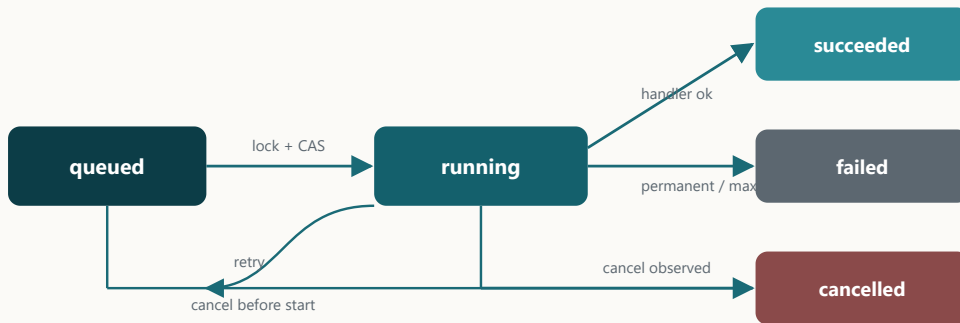


Figure 2. Job status. Conditional updates (`findOneAndUpdate` with expected status) make each arrow single-winner.

The actual update is `findOneAndUpdate({ _id, status: expected }, { $set: { status: next } })`. A null result means another actor won. The worker reloads and stops — usually by completing the Service Bus message so it is not retried.

API

Method	Path	Contract
POST	/jobs	{ type, payload, webhookUrl?, idempotencyKey? } → 202 { jobId, status: "queued" }. Same idempotency key returns the original job. Over cap → 429.
GET	/jobs/:id	Current document. This is status. Clients poll it. Near real time for a single job does not require a second store.
POST	/jobs/:id/cancel	Sets <code>cancelRequestedAt</code> . 202 if <code>queued</code> or <code>running</code> . 409 if already terminal, so the client does not believe cancel took effect.

Submit order: insert `queued`, send the message, return. If the send fails, a sweeper re-enqueues `queued` jobs older than about 15 seconds. The document is never deleted on enqueue failure.

Data model

```
jobs
  _id, tenantId, type, status
  attempt, maxAttempts
  idempotencyKey // unique per tenant
  payloadUri, resultUri
  lastError
  webhookUrl
  cancelRequestedAt
  createdAt, updatedAt
  startedAt, finishedAt
```

```
outbox
  _id, jobId, payload
  availableAt, attempts
  deliveredAt

Service Bus message
  { jobId, type, attempt }
  MessageId = jobId:attempt
```

Indexes: { tenantId, status }, { status, updatedAt } for the sweeper, unique { tenantId, idempotencyKey } when a key is present. Duplicate detection on Service Bus drops a double send of the same attempt.

Flows that matter

Submit and run

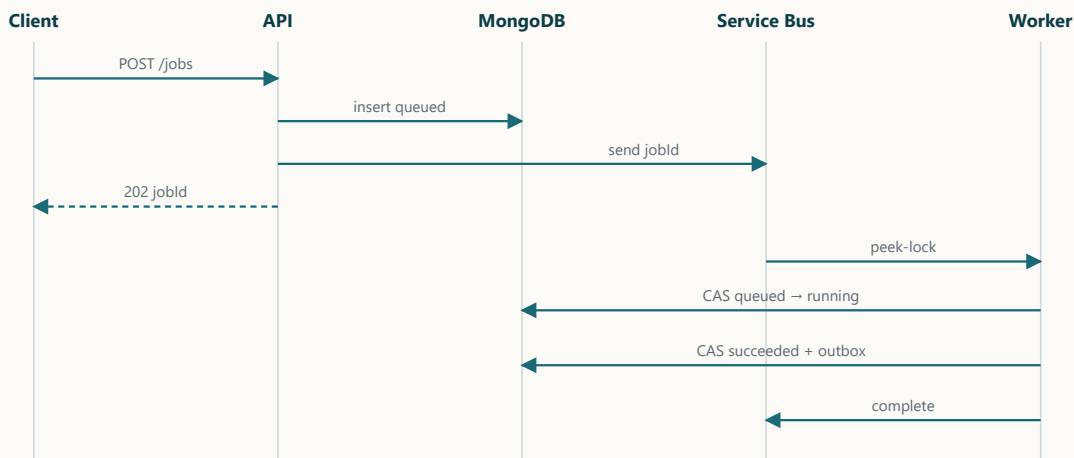


Figure 3. Submit returns before work starts. The worker is the only actor that moves the job to running.

Retry

Retryable errors (timeouts, 503): CAS running → queued, increment attempt, schedule the next Service Bus message with exponential backoff plus jitter ($\min(2^{\text{attempt}}, 300)$ seconds), complete the current message so it does not redeliver immediately. Permanent errors or attempt $\geq$ maxAttempts: CAS to failed plus outbox. Business-validation failures are failed with structured lastError. The dead-letter queue is for poison messages and operator replay, not expected business errors.

Cancel

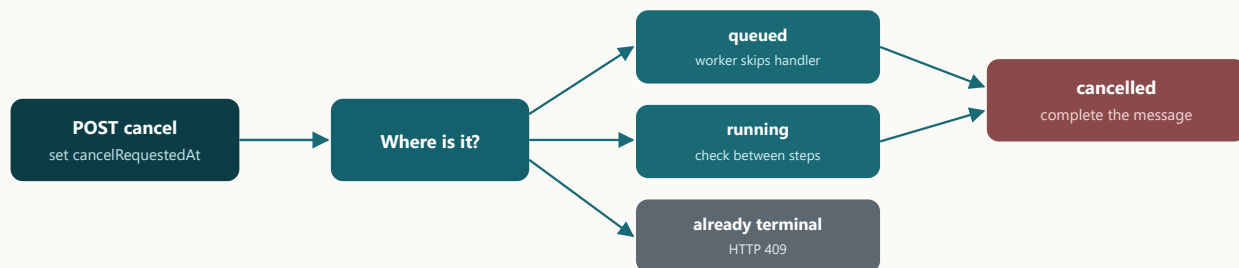


Figure 4. Cancel is a flag on the document. Service Bus cannot pull a locked message back; only the locking worker completes it.

Completing after cancel is required. Abandoning would redeliver a cancelled job. On lock renewal, re-read the flag. Honest constraint: a job may finish the current step after cancel is requested. That is the contract.

Notify

The worker writes terminal status and the outbox document in one MongoDB transaction. A dispatcher claims the outbox row and POSTs a signed webhook. Failures retry the webhook only — never the job. Extra caches are not required for v1; status is GET /jobs/:id.

The hard problems

1. Cancel while the worker already holds the lock

Service Bus cannot take a locked message back. Killing the pod is worse: the lock expires, the message comes back, and you may run cancelled work. Cancel writes `cancelRequestedAt`. If the job is still queued, the worker sees the flag, marks `cancelled`, completes the message, and does not run. If it is running, the handler checks the flag between steps and does the same. If it is already done, return 409.

2. Spikes and noisy-neighbor tenants

The bus is a buffer, not an infinite one.

1. Scale workers on queue depth, with a max replica count so we do not melt MongoDB or downstream APIs.
2. Reject submit with 429 + Retry-After when global depth or that tenant's open jobs (queued + running) exceed a cap.
3. Cap in-flight jobs per worker pod so one slow handler type cannot pin a replica.

3. Retries without double-applying, and status that is actually queryable

The same job can run twice (lock timeout, duplicate enqueue). Idempotent handlers, not hope, are the fix. Conditional status updates plus Service Bus duplicate detection (`MessageId = jobId:attempt`) stop two workers from both starting the same attempt. Status is a MongoDB read. For a single job that is enough. If status traffic later dwarfs writes, add a cache then — not on day one.

Failure modes

Failure	What happens
API dies after insert, before enqueue	Sweeper re-enqueues leftover queued jobs.
Worker dies mid-handler	Peek-lock expires, message comes back. Conditional update stops a double start. A stale running document is returned to queued after a lease timeout on startedAt.
Service Bus down	Submit fails after insert; sweeper retries. GET /jobs/:id still works.
Webhook endpoint down	Outbox retries. Job stays terminal. The client can still read status.
Poison payload	Max delivery count → dead-letter queue; job failed.

How I would structure the NestJS code

One repo, two processes, shared modules:

- `JobsModule` — HTTP submit, status, cancel.
- `WorkerModule` — Service Bus receiver, handler registry, lock renewal, cancel checks.
- `JobsRepository` — `findOneAndUpdate` plus outbox in one transaction.
- `NotificationModule` — outbox poller, webhook client.

```
interface JobContext {
  jobId: string;
  tenantId: string;
  attempt: number;
  payload: unknown;
  isCancelled(): Promise<boolean>; // read cancelRequestedAt
}
```

`isCancelled()` is the cooperative interrupt. A handler that ignores it cannot be cancelled mid-execution; that is documented, not papered over.

What I would not do yet

- **Run the job inside the HTTP request.** It cannot last minutes and cannot absorb a spike.
- **Use MongoDB as the queue** (a work collection you poll). Fine at small scale. Service Bus is there for spikes, DLQ, and scheduled retry.
- **Redis, Kafka, a second queue, or a workflow engine.** This prompt is submit / run / retry / status / cancel. A queue plus a job collection covers it. Add a cache when status reads are actually hot. Add a workflow engine if jobs later wait on humans for hours.

Build the job collection and the queue first. Everything else is a scaling add-on you can name when asked, not something you have to operate in v1.